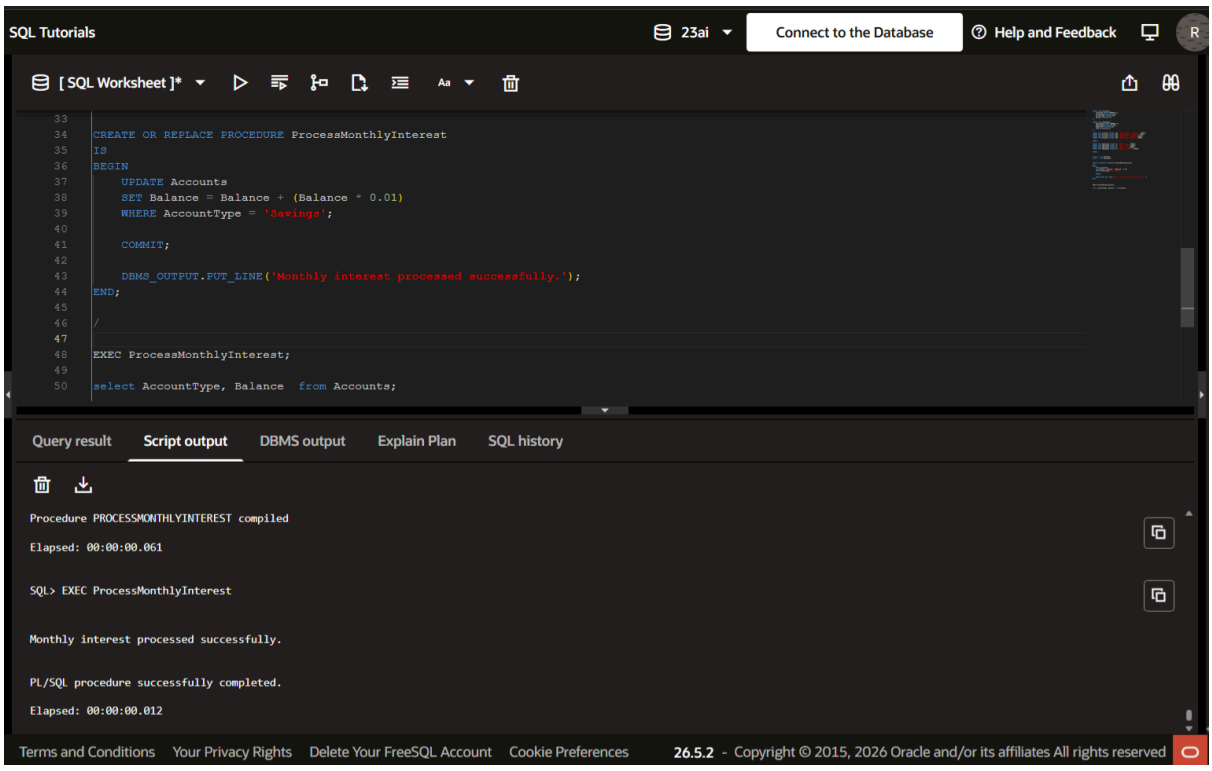
```
CREATE TABLE Accounts (
   AccountID NUMBER PRIMARY KEY,
   CustomerName VARCHAR2(100),
   AccountType VARCHAR2(20),
   Balance NUMBER(12,2)
);

CREATE TABLE Employees (
   EmployeeID NUMBER PRIMARY KEY,
   EmployeeName VARCHAR2(100),
   Department VARCHAR2(50),
   Salary NUMBER(12,2)
);

INSERT INTO Accounts VALUES (101, 'Rahul Sharma', 'Savings', 10000);
INSERT INTO Accounts VALUES (102, 'Priya Singh', 'Savings', 5000);
INSERT INTO Accounts VALUES (103, 'Aman Das', 'Current', 20000);
INSERT INTO Accounts VALUES (104, 'Neha Roy', 'Savings', 15000);

COMMIT;

INSERT INTO Employees VALUES (1, 'Rahul', 'IT', 50000);
INSERT INTO Employees VALUES (2, 'Priya', 'HR', 45000);
INSERT INTO Employees VALUES (3, 'Aman', 'IT', 60000);
INSERT INTO Employees VALUES (4, 'Neha', 'Finance', 55000);

COMMIT;


select * from Accounts;
select * from Employees;
```

Scenerio 1

o **Question:** Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
IS
BEGIN
   UPDATE Accounts
   SET Balance = Balance + (Balance * 0.01)
   WHERE AccountType = 'Savings';

   COMMIT;

   DBMS_OUTPUT.PUT_LINE('Monthly interest processed successfully.');
END;

/

EXEC ProcessMonthlyInterest;
```

OUTPUT

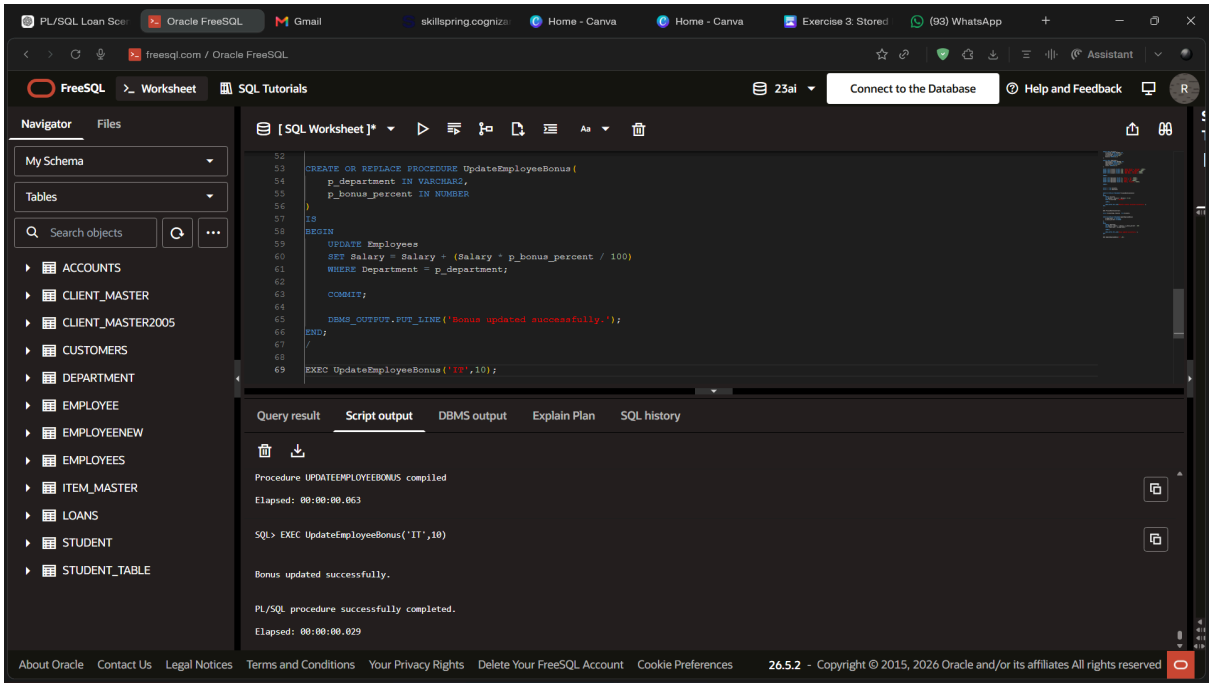


Scenerio 2

o **Question:** Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
   p_department IN VARCHAR2,
   p_bonus_percent IN NUMBER
)
IS
BEGIN
   UPDATE Employees
   SET Salary = Salary + (Salary * p_bonus_percent / 100)
   WHERE Department = p_department;

   COMMIT;

   DBMS_OUTPUT.PUT_LINE('Bonus updated successfully.');
END;
/

EXEC UpdateEmployeeBonus('IT',10);
```

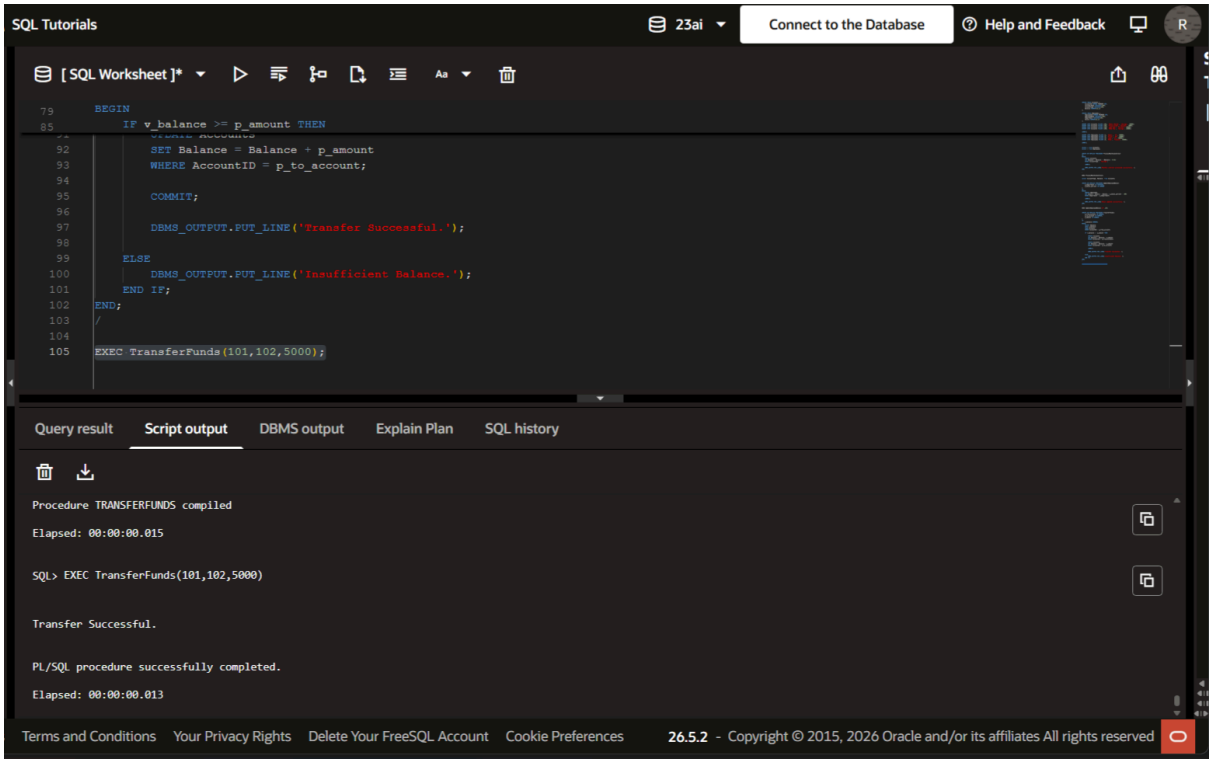
OUTPUT



Scenerio 3

- **Question:** Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

```
CREATE OR REPLACE PROCEDURE TransferFunds(
    p_from_account IN NUMBER,
    p_to_account IN NUMBER,
    p_amount IN NUMBER
)
IS
    v_balance NUMBER;
BEGIN
    SELECT Balance
    INTO v_balance
    FROM Accounts
    WHERE AccountID = p_from_account;

    IF v_balance >= p_amount THEN

        UPDATE Accounts
        SET Balance = Balance - p_amount
        WHERE AccountID = p_from_account;

        UPDATE Accounts
        SET Balance = Balance + p_amount
        WHERE AccountID = p_to_account;

        COMMIT;

        DBMS_OUTPUT.PUT_LINE('Transfer Successful.');

    ELSE
        DBMS_OUTPUT.PUT_LINE('Insufficient Balance.');
    END IF;
END;
/

EXEC TransferFunds(101,102,5000);
```


SQL Tutorials
23ai
Connect to the Database
Help and Feedback
[SQL Worksheet]*
BEGIN
IF v_balance >= p_amount THEN
SET Balance = Balance + p_amount
WHERE AccountID = p_to_account;
COMMIT;
DBMS_OUTPUT.PUT_LINE('Transfer Successful.');
ELSE
DBMS_OUTPUT.PUT_LINE('Insufficient Balance.');
END IF;
END;
/
EXEC TransferFunds(101,102,5000);
Query result
Script output
DBMS output
Explain Plan
SQL history
Procedure TRANSFERFUNDS compiled
Elapsed: 00:00:00.015
SQL> EXEC TransferFunds(101,102,5000)
Transfer Successful.
PL/SQL procedure successfully completed.
Elapsed: 00:00:00.013
Terms and Conditions
Your Privacy Rights
Delete Your FreeSQL Account
Cookie Preferences
26.5.2 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved

OUTPUT- 3